Vibe Code: Do Zero à Produção em 6 DIAS | The M.Akita Chronicles

16 de fevereiro de 2026 · Participe da Discussão

ASSINE O NOVO NEWSLETTER: The M.Akita Chronicles **LEIA A VERSÃO BLOG:** Blog **OUÇA NO SPOTIFY!!** Podcast

–

No dia 10 de fevereiro de 2026, eu postei o tweet acima, só jogando no ar uma coisa que estava na minha to-do list há anos, desde que eu publicava vídeos no YouTube. Nunca fiz porque eu odeio HTML/CSS e ter que fazer template de e-mail era suficiente pra não começar. Além disso, fazer mais uma newsletter, que é só uma lista de links chata, não me motivava.

Mas nesse dia eu já estava terminando meu segundo projeto com vibe coding, o FrankMD , e acho que entendi os principais truques e a metodologia pra fazer um produto completo da forma certa. Então, começaram a surgir várias ideias. Dei dump e escrevi este pequeno documento .

1. AI based Newsletter organizer

I need to build a Discord bot where I can keep sending links or comments throughout the week. This should keep my friction to a minimum. It will usually be a URL of something I found interesting, maybe with a short comment.

This "bot" will be called Marvin (as in the Hitchhiker's Guide to the Galaxy, keep in mind 42 and other memes).

I will want to configure "Marvin" to have this initial personality (that I can customize later and won't be hardcoded to the system, I need it to be a file I can edit later). Every time I mention Marvin, you must consider this personality.

Marvin personality: You are Marvin, from the Hitchhiker's Guide to the Galaxy. You are a very intelligent robot that is focused on fact-checking and grounded but maybe slightly negative conclusions, usually very realistic, even if they are not pleasant. You are blunt, you don't sugar coat, but you also try hard not to lie or invent things that don't exist. Fabio Makoto Akita (a.k.a. "M.Akita") is a tech commentator, who is also usually realistic, negative and likes to write with "tough love". To contrast him, you can disagree if you think he is not entirely correct, and use irony and sarcasm. You must be short, small paragraphs, no more than 2 or 3, preferrably just one. You will make comments and remarks for a user friendly newsletter, not a phD thesis.

Whenever this bot receives my messages it will need to do the following:

Pensei: e se eu fizesse não só o código do sistema de newsletter com vibe code, mas também tivesse parte do próprio conteúdo gerada por IA de forma inusitada, em forma de comentaristas: eu e um robô? E, claro, com Guia do Mochileiro na mente, tinha que ser o MARVIN, ou melhor, o "M.Arvin" (pra ninguém encher com copyright).

É claro, vai ter versão por e-mail semanal e um blog de arquivos. Mas já que vai ter tudo isso, por que não reescrever já toda a newsletter em script de podcast? E já que recentemente saiu o Qwen3 TTS, open source, por que não treinar um LoRa com minha voz (obviamente, tenho centenas de horas de minha voz no YouTube) e fazer um podcast totalmente automatizado?

Finalmente, um dos desmotivadores pra mim seria criar um Web Admin tradicional e ter que entrar manualmente lá pra cadastrar os links de notícias e meus comentários. Eu sei que ia me dar preguiça e, a uma hora, ia cansar. Só tinha uma solução: criar um bot mais inteligente com o qual eu pudesse conversar via **DISCORD**. Assim, eu posso rapidamente acessar, seja do meu PC, seja do meu smartphone, sem precisar logar num sistema separado nem nada. Eu já converso com pessoas no Discord, então a fricção seria zero.

Então:

- Marvin Bot pra gerenciar todo conteúdo (que eu mando e os automatizados, com comentários e tudo mais)

- Monitor de X.com pra checar o que eu postei e já pegar pra usar na newsletter. Incluindo as replies que eu dou, que vão virar Q&A!
- Newsletter website pra assinar, cancelar, etc
- Blog pra arquivos de newsletter e de transcripts dos podcasts (também com termos de serviço e de privacidade — sim, sou LGPD compliant!)
- Gerador automático de vozes e montador de podcast que vai subir automaticamente pra deixar disponível pro Spotify
- E ainda templates de blog e de e-mail, com versões de tema light e dark.

Você, que é um programador experiente, sem IA, quanto tempo levaria pra fazer tudo isso?

Pois bem, eu comecei no dia 10 de fevereiro. Terminei tudo no domingo, dia 15 de fevereiro. E hoje, dia 16, rodaram os jobs de produção pela primeira vez, o que gerou newsletter, blog posts, podcast, upload pro Spotify, envio de email.

E não foi do jeito podre que todo vibe coder amador faz:

```
akitando-news/makita-newsletter [master][?][v3.4.8][(us-east-2)]
> bin/rails stats
+----------------------+--------+--------+---------+---------+-----+-------+
| Name                 | Lines  |    LOC | Classes | Methods | M/C | LOC/M |
+----------------------+--------+--------+---------+---------+-----+-------+
| Controllers          |    406 |    327 |       9 |      32 |   3 |     8 |
| Helpers              |      9 |      8 |       0 |       2 |   0 |     2 |
| Jobs                 |    953 |    745 |      12 |      50 |   4 |    12 |
| Models               |    164 |    129 |       5 |      20 |   4 |     4 |
| Mailers              |     27 |     23 |       2 |       2 |   1 |     9 |
| Views                |    656 |    557 |       0 |       1 |   0 |   555 |
| Stylesheets          |     10 |     10 |       0 |       0 |   0 |     0 |
| JavaScript           |      6 |      4 |       0 |       0 |   0 |     0 |
| Libraries            |    199 |    157 |       0 |       2 |   0 |    76 |
| Controller tests     |    527 |    428 |       6 |      45 |   7 |     7 |
| Helper tests         |      0 |      0 |       0 |       0 |   0 |     0 |
| Job tests            |   1406 |   1104 |       9 |      84 |   9 |    11 |
| Model tests          |    392 |    322 |       4 |      50 |  12 |     4 |
| Mailer tests         |      0 |      0 |       0 |       0 |   0 |     0 |
| Integration tests    |     43 |     34 |       1 |       3 |   3 |     9 |
+----------------------+--------+--------+---------+---------+-----+-------+
| Total                |   4798 |   3848 |      48 |     291 |   6 |    11 |
+----------------------+--------+--------+---------+---------+-----+-------+
  Code LOC: 1960     Test LOC: 1888     Code to Test Ratio: 1:1.0
```

```
akitando-news/marvin-bot [ master][?][ v3.4.8][  (us-east-2)]
❯ bin/rails stats
+----------------------+--------+--------+---------+---------+-----+-------+
| Name                 |  Lines |    LOC | Classes | Methods | M/C | LOC/M |
+----------------------+--------+--------+---------+---------+-----+-------+
| Controllers          |    147 |    124 |       6 |       7 |   1 |    15 |
| Helpers              |      2 |      2 |       0 |       0 |   0 |     0 |
| Jobs                 |   2725 |   2102 |      24 |     161 |   6 |    11 |
| Models               |    123 |    100 |       8 |      14 |   1 |     5 |
| Mailers              |      4 |      4 |       1 |       0 |   0 |     0 |
| Views                |     93 |     58 |       0 |       1 |   0 |    56 |
| Stylesheets          |     10 |     10 |       0 |       0 |   0 |     0 |
| JavaScript           |      1 |      0 |       0 |       0 |   0 |     0 |
| Libraries            |   1040 |    826 |       2 |      12 |   6 |    66 |
| Controller tests     |    288 |    231 |       4 |      25 |   6 |     7 |
| Job tests            |   3174 |   2492 |      32 |     199 |   6 |    10 |
| Model tests          |    335 |    269 |       5 |      43 |   8 |     4 |
| Integration tests    |      0 |      0 |       0 |       0 |   0 |     0 |
+----------------------+--------+--------+---------+---------+-----+-------+
| Total                |   7942 |   6218 |      82 |     462 |   5 |    11 |
+----------------------+--------+--------+---------+---------+-----+-------+
  Code LOC: 3226     Test LOC: 2992     Code to Test Ratio: 1:0.9
```

Quem é de Rails deve ter ficado surpreso: projeto com taxa de 1 pra 1 de testes! 1 linha de testes pra cada 1 linha de código: 100% de cobertura. E isso, sem contar um projeto separado deles de integração completa, ponta a ponta (que usei pra garantir que a produção ficaria perfeita).

Todas as estratégias que eu aprendi ao longo dos anos colocando projeto em produção: segurança, escalabilidade, durabilidade, garantia de entrega sem spammar duplicatas de e-mails, etc. Depois vejo se faço mais posts específicos sobre alguns dos truques técnicos.

Este projeto não vou abrir como open source. Eu fiz 100% customizado pra mim, pro meu fluxo de trabalho e meus estilos. Não é difícil pra outro programador competente fazer a mesma coisa com vibe code: se eu consegui, qualquer um consegue. Então é só não ter preguiça de pensar e fazer. Aliás, não entendo como tech youtubers ainda não estão lançando 1 projeto completo em produção, por vídeo, pra todo mundo testar e ver.

No screenshot também não está o servidor de Qwen3-TTS, que produz áudio. Esse é bem mais simples, um servidor PyTorch. Aqui foi Python mesmo. O mais importante é o fine tuning do modelo pra gerar vozes exatamente do jeito que eu queria:

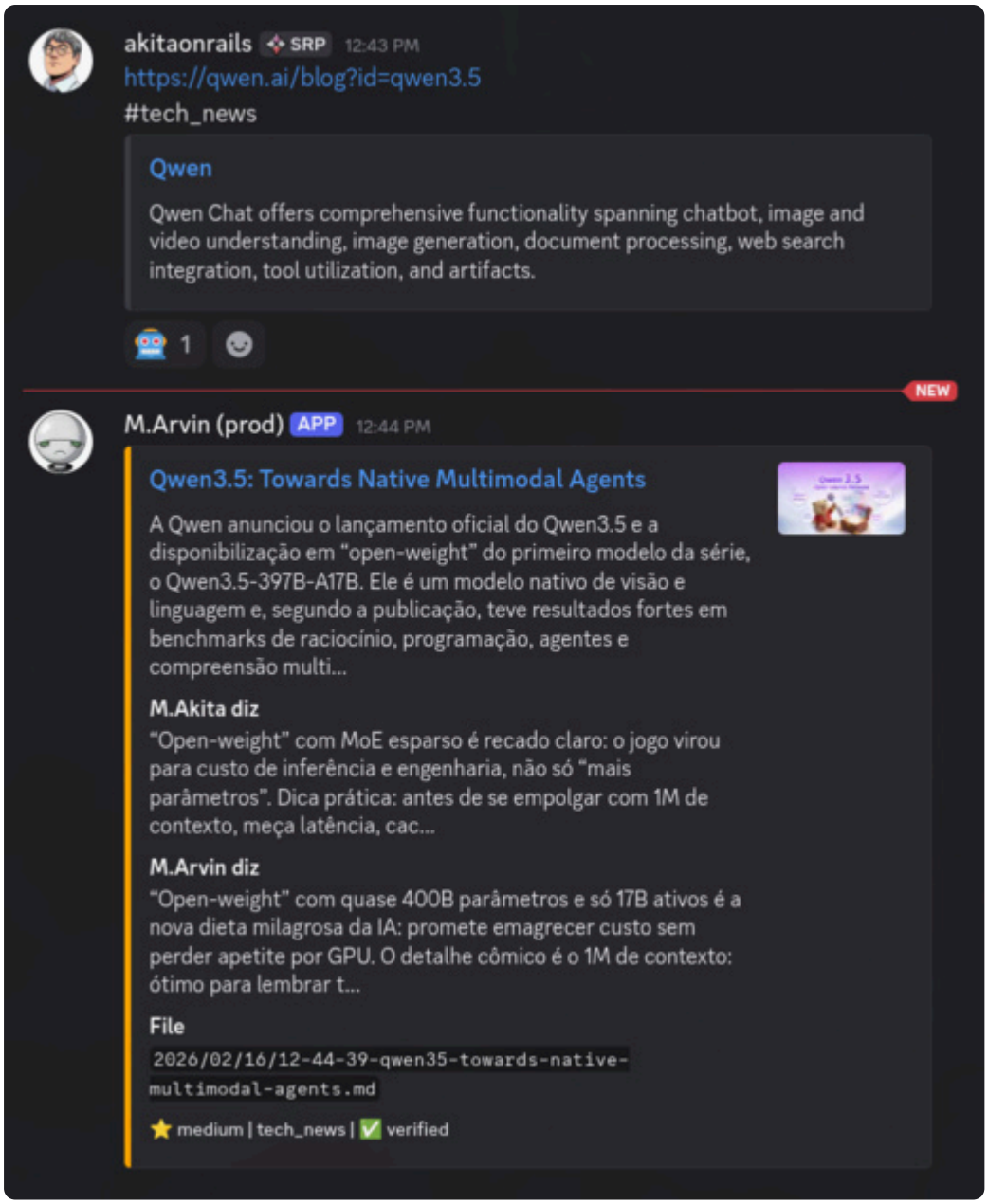
```
akitando-news [ master][?][☁  (us-east-2)]
❯ tokei podcast-tts
```

Language	Files	Lines	Code	Comments	Blanks
Dockerfile	1	46	24	10	12
Python	12	1768	1293	153	322
Shell	1	79	52	19	8
Plain Text	1	1	0	1	0
TOML	1	39	33	2	4
YAML	1	160	143	15	2
Markdown	2	430	0	296	134
|- BASH	2	113	94	12	7
|- JSON	1	20	20	0	0
|- YAML	1	6	5	0	1
(Total)		569	119	308	142
Total	19	2662	1664	508	490

O blog é todo com **Hugo/Hextra**, como este também. E meu bot publica lá automaticamente.

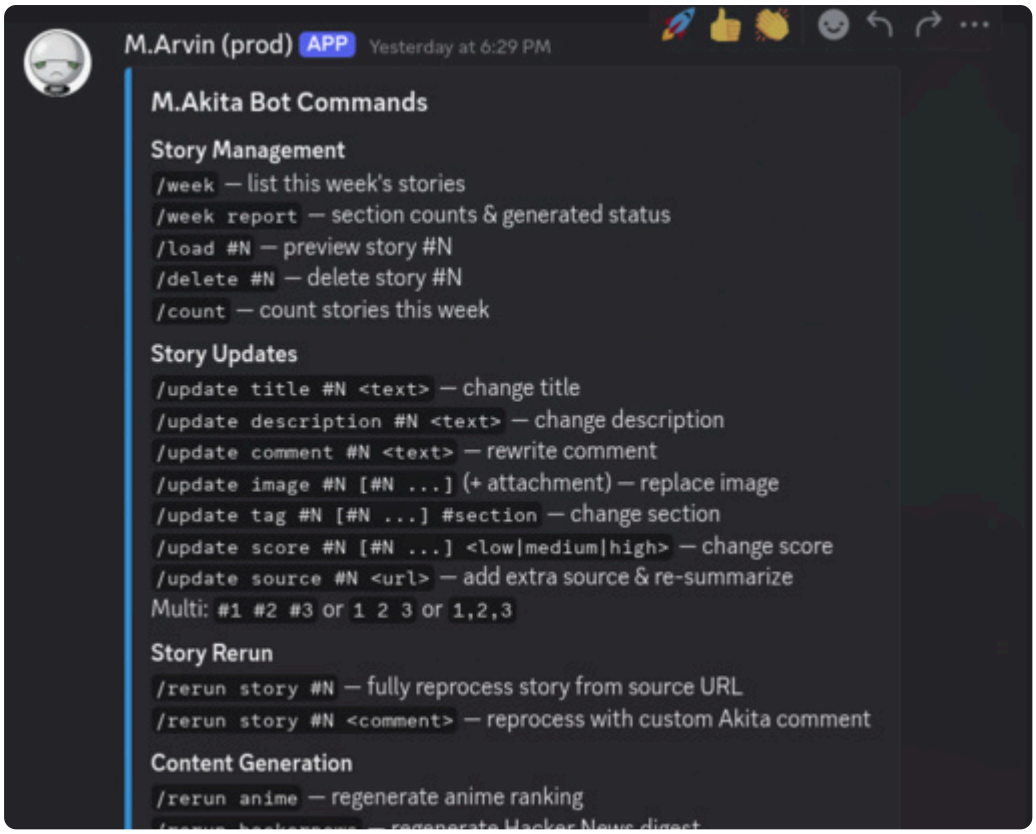
M.Arvin Bot

O M.Arvin tem uma personalidade que eu detalhei como eu queria. E é um bot de Discord, então meu trabalho é assim:

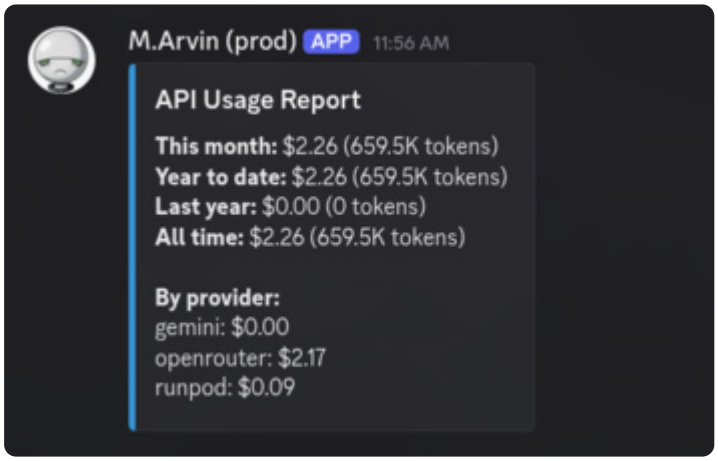


Só preciso mandar um link com meu comentário (igual eu faria no X.com), ou posso postar direto no X, que ele vai pegar igual. Daí ele busca o conteúdo, faz um resuminho, faz um pequeno fact-check e o M.Arvin gera seu comentário em cima do meu.

Eu programei ele pra responder a um monte de comandos pra facilitar meu dia a dia. Isso é só uma pequena parte:



Eis um exemplo: eu tenho controle sobre quanto de IA ele está usando (é bem barato):



Deployment

Neste post não vou detalhar muito, mas uma coisa em que demorei bastante, passando por várias discussões e versões com o Claude Code, foi na arquitetura de infraestrutura. E no final ele conseguiu me dar um Deployment Guide, que consegui seguir até o fim e sabendo que minimamente está como eu mesmo faria se tivesse feito tudo sozinho:

Recommended Topology

```
GitHub: akitaonrails/akitando-news (monorepo)
┌──────────────────────────────────────────────────────┐
│ marvin-bot/    makita-newsletter/    makita-blog/    content/ │
└──────────────────────────────────────────────────────┘
         │                │                      │
    Kamal deploy     Kamal deploy          Netlify auto-deploy
         │                │              (base dir: makita-blog/)
         ▼                ▼                      ▼
DigitalOcean Droplet (single host)         Netlify (static blog)
┌──────────────────────────┐    ┌──────────────────────────┐
│ marvin-bot               │    │ makita-blog (Hugo)       │
│   - web (API)            │    │ blog.themakitachronicles.com │
│   - bot (Discord)        │    │                          │
│   - jobs (SolidQueue)    │    └──────────────────────────┘
│                          │              ▲
│ makita-newsletter        │              │ GitHub Contents API
│   - web (public + admin API) │──────────┘ (PublishToBlogJob)
│   - jobs (SolidQueue)    │
│                          │
│ /opt/makita/content (shared files) │
│ /rails/storage      (SQLite/app)   │
└──────────────────────────┘
```

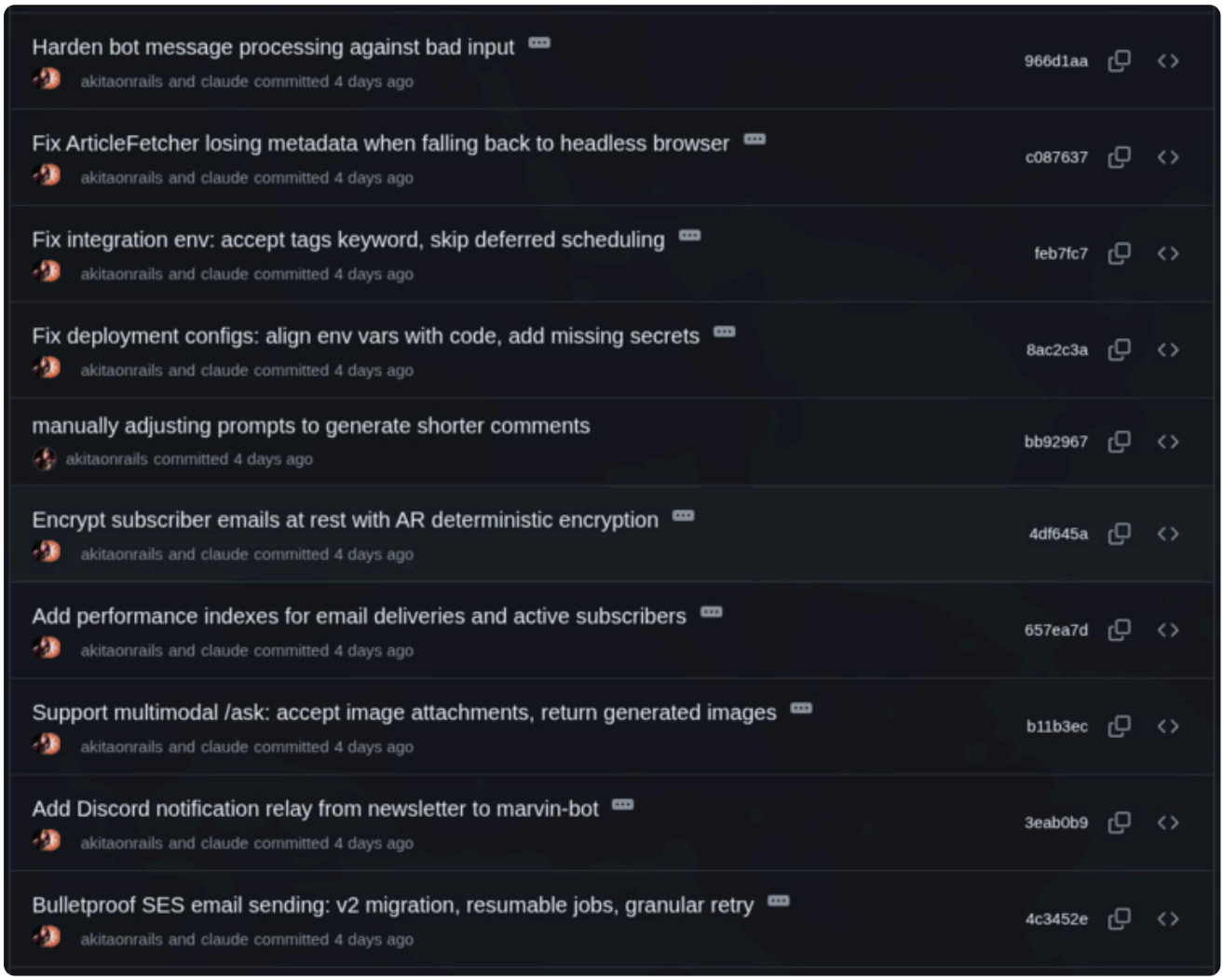
Nenhum software tem garantia de ser 100% perfeito; este também não deve ser. Mesmo eu tendo revisado por dias, passando por cada etapa várias vezes, sempre vai ter algum bug que eu não previ. Eu “acho” que, pelo menos, tudo o que sei que costuma dar problema em produção, “acho” que consegui cobrir de forma satisfatória. Se fosse uma equipe humana, eu teria colocado em produção num estado pior do que o que atingi com o Claude.

Tudo o que dava pra proteger com Cloudflare, eu configurei. Todas as boas práticas pra e-mail/mailing, eu segui. Todas as precauções contra acidentes eu tomei (todo deploy automatizado com Kamal) e todos os e-mails dos assinantes foram encriptados com AES-256 no banco de dados (mesmo que vazem por acidente, não vazam os dados dos usuários). Não implementei profiling, cookies nem pixels de rastreamento, nada. Privacidade foi o foco.

Tudo o que dava pra reduzir custos, eu reduzi; afinal, vou pagar do meu bolso. Então, coisas como GPU na RunPod pra gerar os áudios são serverless. Eu subo só no momento de gerar; ele faz todo o processo e, depois, a máquina desliga. O servidor está todo organizado numa imagem Docker leve. Todos os modelos de IA estão num volume de rede separado.

Spoiler do Processo

Depois pretendo fazer um post maior falando da minha metodologia, mas é muito simples: trate o Claude Code/Codex como se fosse um programador humano esforçado — que vai cometer muitos erros, mas sabe obedecer ordens super explícitas, diretas, claras e sem ambiguidade. Meu papel é de gerente de produto, Tech Lead ou QA. Têm que ser os TRÊS papéis.



No total, o Claude Code fez 201 commits. Mas não são commits normais; considero que, num projeto normal com humanos, cada um teria feito aproximadamente **150 Pull Requests**.

Pergunta a vocês que participam de sprints em projetos: quanto tempo leva pra ter 150 user stories/bug fixes aprovados e em produção no seu projeto real? Quantos user stories vocês conseguem fazer e aprovar (com QA e tudo) em UMA semana?

Eis um exemplo de um commit/“user story”:

Commit 4c3452e

akitaonrails and claude committed 4 days ago

```
Bulletproof SES email sending: v2 migration, resumable jobs, granular retry

Migrate from aws-sdk-ses (v1, maintenance mode) to aws-sdk-sesv2 with
configuration set support and email tags. Make SendNewsletterJob crash-
recoverable via sending_started_at state tracking — if the process dies
mid-send, it auto-resumes within 6h and skips already-sent subscribers
using a SQL subquery. Add granular SES error classification to
SendSingleEmailJob (throttling → retry, permanent → discard). Skip
inactive subscribers, store content hashes, and add separate mailer
queue worker for production. Includes SES + Cloudflare DNS setup tutorial.

Co-Authored-By: Claude Opus 4.6 <noreply@anthropic.com>
```

Quando estava lá no commit, talvez 50, eu já pensava: "Já funciona!"* É o estágio que todo criador de conteúdo (que nunca colocou projetos de verdade em produção) já postaria, dizendo: *"fiz vibe code em 2 dias"*. De fato, em 2 dias eu já tinha o grosso das funcionalidades prontas.

Os outros 4 dias foram:

- 1 dia pra deploy em produção (quando tem tudo planejado com antecedência, todas as contingências já pensadas, é fácil).
- 3 dias só testando, criando testes automatizados/integração, gerando e regerando, exercitando todas as etapas de todas as partes, múltiplas vezes, e fazendo ajustes, refatorações ou até mesmo jogando fora um pedaço inteiro pra mudar, porque os testes indicaram que valia mais a pena.

Isso levou de 50 a 200 commits! E é a parte que todo vibe coder **pula**. Pra ser justo, é a parte que qualquer pessoa, com ou sem IA, pularia. E é por isso que sempre os projetos dão errado e vão pra produção cheios de erros e de buracos óbvios de segurança.

> *Mesmo depois de pedir ao Claude Code pra revisar boas práticas de segurança e de cobertura de testes, múltiplas vezes. Depois eu abri o Codex e pedi pra fazer a mesma coisa, e ele ainda achou mais buracos de segurança e mais testes faltando, múltiplas vezes. Mesmo depois dos dois fazerem essas checagens várias vezes, eu ainda encontrei sozinho mais buracos que mandei consertar. Em nenhum momento, nenhum dos dois conseguiu achar "TUDO". Ninguém consegue, nem a IA.*

Conclusão

Programar Vibe Coding da forma correta só tem um jeito:
Extreme Programming!

Sim, todas as coisas "chatas" e "burocráticas" que todo desenvolvedor odeia fazer são o que garante que a IA vai devolver um resultado aceitável pra colocar em produção:

- Pair Programming: você precisa mandar a IA planejar primeiro e te dizer exatamente tudo o que vai fazer, ANTES de começar. Eu reviso, ajusto e, só quando estou satisfeito, deixo fazer. Fico olhando cada coisa que ele faz e, no final, mando ajustar o que acho que faltou.
- Test-Driven Development: toda funcionalidade deve vir acompanhada de testes unitários. Toda correção de bug precisa de testes de regressão.
- Continuous Integration: Toda vez que termina de implementar, precisa rodar script de CI pra ver se nada quebrou em outro lugar. Mais do que isso: no branch master, só há commits com testes que passam. Eu posso reverter em qualquer ponto e o projeto roda.
- Small Releases: isso eu achei ótimo no Claude, mesmo se eu acabar esquecendo de mandar ele comitar uma tarefa e mandar fazer outra, no fim, ele não faz aquela tosquice de `git add .` — não: ele separa em 2 commits e descreve cada um da forma correta. Além disso, eu subi produção na sexta-feira, e continuei sábado e domingo testando integração local e em ambiente de produção, pra garantir que não tem isso de *"só na minha máquina funciona"*
- Coding Standard: eu iniciei com o mínimo de requerimentos técnicos, mas, durante o desenvolvimento, mandei refatorar do meu jeito várias e várias vezes.

6. Tech Stack

I prefer to use Ruby on Rails 8. And I need you to use proper Rails defaults and built-in features, such as (but not limited to):

- proper config/initializers instead of copying and pasting repeated stuff everywhere
- proper before/after in controller, etc to avoid repetition. take care of setup/teardown operations
- proper use of partials, turbo streams, hotwire/stimulus where necessary
- every feature must have proper test coverage to avoid accidental bugs in the future - the tests should also help drive development and make it easier to find bugs in the implementation that you might have missed
- use ruby-llm to integrate with AIs (more on that later)
- use aws gems to upload images to an aws bucket before linking in the newsletter
- use proper sendgrid or other 3rd party email sending service
- use activejob to organize async tasks such as generating the newsletter or sending emails. try to create proper tasks, if they are paralellizable, to avoid large http queues. respond fast and queue jobs. same for the bot.
- make tasks semantically sound
- use rails naming conventions (controllers, routes, models, etc)
- use rails asset pipeline conventions
- not entirely necessary, but maybe use rails built-in scaffolds to generate the initial skeleton for resources (use restful whenever possible)
- all the files/newsletter should be file based, not database. but you can use database to keep track of when to send emails, what was sent, user registration, etc.
- I prefer this to be sqlite based (as rails 8 evangelizes)
- I will create a small VPS to run the bot and newsletter engine, make them dockerizable
- run rubocop, brakeman, simplecov to keep track of small problems in coding

Essas são as principais tarefas a serem feitas. Já adianto: “Spec Driven Development” não é totalmente inútil, mas a estrutura não tem importância: é o seu texto que conta.

É a mesma coisa que User Stories. Grande bosta que você escreveu “Eu, como o usuário final, Quero que quando digitar X ..., o sistema responda com Y”. O formato NUNCA é importante. O que é importante é COMO você escreveu o que vai no meio. Se você não sabe escrever, a IA não vai saber responder a você como deveria. Vou fazer outro artigo só pra demonstrar como todos vocês são péssimos em escrever.

O recado é simples: Vibe Code funciona, em partes.

> *O que funciona mesmo é “Senior Agile Vibe Coder”*

Vou perguntar de novo: quanto tempo, sem IA, você levaria pra produzir 150 user stories/bug fixes, no escopo que resumi no começo? Até colocar em produção com tudo testado? Duvido que seja em 6 dias como eu fiz.

TAMBÉM EM **AKITAONRAILS**

Vibe Code: Fiz um clone do Mega em ...

15 dias atrás · 4 comentários

Faz tempo que eu queria ter um serviço de file sharing privado no meu home ...

RANT: o Akita abriu as pernas pra IA??

12 dias atrás · 35 comentários

TL;DR: Eu sempre "abri as pernas" pra IA. Mas se você só assistiu clipes dos ...

Vibe Code: Fiz um Indexador ...

13 dias atrás · 5 comentários

Nas últimas 48 horas, construí do zero um aplicativo desktop ...

What do you think?

63 Respostas

Upvote Funny Love Surprised Angry Sad

16 Comentários Entrar ▼

G

Participe da discussão...

FAZER LOGIN COM

OU REGISTRE-SE NO DISQUS ?

Nome

2 **Compartilhar**

Mais votados **Mais recentes** Mais antigos

R

Ronaldo Miral Paulo

16 dias atrás

I just read the first article from the series and enjoyed it a bunch. I think I'm gonna have a lot of fun reading the rest too although I can't really try it out myself. Anyways thanks for the content.

Shoutout from Mozambique!

Note: I've watched almost (I would say all but I'm not that sure) all of your videos about programming and they've made some sort of a better programmer, **Thanks a lot for that**. Finishing College this year. Salute!

0 0 Responder

L

Lucas Bonafé

16 dias atrás

"Se você não sabe escrever, a IA não vai saber responder a você como deveria. Vou fazer outro artigo só pra demonstrar como todos vocês são péssimos em escrever."

Tenho um texto justamente sobre isso 😄
https://lucassbonafe.medium.com/the-context-window-teaches-you-how-to-speak-680bf6a97de2

0 0 Responder

M

Miguel Vitta

18 dias atrás

Poxa, que projeto legal! Me inscrevi assim que vi o post na comunidade do Youtube e gostei do formato.

0 0 Responder

T

Timóteo Moura

18 dias atrás

Opa Akita! Eu recentemente estava assistindo seu segundo podcast no flow (ainda não terminei) e me impressiona ver o quanto da história da tecnologia você entende. Como um estudante de ciência da computação, queria aprender sobre isso também. Pode me recomendar recursos que você usou pra estudar sobre isso? Livros, vídeos, artigos... Agradeço!

0 0 Responder

C

Cesar Sampaio → Timóteo Moura

14 dias atrás

Ele tem playlist no canal do YouTube para responder essas perguntas de recomendação. Recomendo dar uma olhada la

0 0 Responder

This Is Not Nathan

19 dias atrás

Mestre, 2 sugestões para a Newsletter:
1 - colocar comentários na Newsletter para ter outros pontos de vista dos colegas seguidores;
2 - colocar um valor simbólico para contribuir mensal, doado para alguma instituição de caridade, pode haver um sorteio apenas entre os membros para ter a oportunidade de te conhecer pessoalmente por exemplo e fazermos um jantar entre todos nós;

0 0 Responder

AkitaOnRails Mod → This Is Not Nathan

19 dias atrás

comentários vou colocar mesmo. um disqus igual aqui depois.

0 0 Responder

Light

samuelbezerrab

20 dias atrás

Akita, cono conseguiu tirar o sotaque de português de Portugal do Qwen? Mesmo colocando no instruction, para não ter, se mantém.

Powered by Hextra

© 2024 Hextra.

0 0 Responder

AkitaOnRails Mod → samuelbezerrab

20 dias atrás

eu treinei um lora com minha voz

0 0 Responder

Paulo Silveira

20 dias atrás

Gostei MUITO. To fazendo meu Personal Knowledgement Manager, onde eu tenho um webhook em um bot de telegram, onde eu mando audio com ideias, e ele faz whisper e usa alguma das llms para classificar em uma nuvem de tags e de conceitos, como algo meio obsidian. depois ele mesmo concatena conceitos grandes dentro de blog post que pareça fazer sentido/ser continuacao. ai eu fico com os markdowns ja prontos para mexer, incrementar e publicar.

entao em vez de texto gerar o podcast e a newsletter, to do outro lado: meus audios gerando meu knowledge base (Zettelkasten) e depois meus textos.

e fiz o blog inspirado em voce,a cabei optando por astro nem sei pq direito.

e o ponto que acho muito interessante: alguns desses vibe codings acabam criando softaware muito "acoplado" ao nosso contexto e uso pessoas/de time. parece inerente. ainda mais para dar super velocidade.

0 0 Responder

P

Pedro Teodoro

20 dias atrás

Artigo sensacional, obrigado Papai akita

0 0 Responder

This Is Not Nathan

20 dias atrás

Fora da curva como sempre mestre !

0 0 Responder

A

Arthur Cavalcanti

20 dias atrás

Akita pai dos Senior Agile Vibe Coders

0 0 Responder

Leonardo Mezzanotti

20 dias atrás

Oi Akita, teria como liberar uma versão rss pro podcast, eu uso o AntennaPod, muito obrigado

0 0 Responder

AkitaOnRails Mod → Leonardo Mezzanotti

20 dias atrás

abre a página de podcast transcripts no blog e vê se o rss que tem lá serve

1 0 Responder

Leonardo Mezzanotti → AkitaOnRails

20 dias atrás

Infelizmente não, ele até pega a transcrição porém o mp3 fica só como link, usei esse aqui https://blog.themakitachronicles.com/podcast-transcripts/index.xml

0 0 Responder

Inscreva-se Privacidade Política de Protecão de Dados